

Secure Data Transmission using Encryption

Tool: OpenSSL / GnuPG

Theory

Encryption converts readable data into an unreadable format that can only be reversed with the correct key, protecting information during storage and transmission. OpenSSL is a command-line toolkit that supports symmetric encryption such as AES, asymmetric encryption such as RSA, and hash generation. GnuPG implements the OpenPGP standard using a key pair where the public key encrypts and the private key decrypts, making it suitable for secure file and message exchange.

Steps

1. Open the terminal in Kali Linux. Verify OpenSSL is installed. Run: `openssl version` — Verify GnuPG is installed. Run: `gpg --version`
 2. Create a sample file to encrypt. Run: `echo 'This is confidential data' > plaintext.txt` — View the file. Run: `cat plaintext.txt`
 3. Encrypt the file using AES-256 symmetric encryption. Run: `openssl enc -aes-256-cbc -pbkdf2 -in plaintext.txt -out encrypted.enc` — Enter a passphrase when prompted and confirm it. View the encrypted file. Run: `cat encrypted.enc` — The output is unreadable binary data.
 4. Decrypt the encrypted file. Run: `openssl enc -aes-256-cbc -d -pbkdf2 -in encrypted.enc -out decrypted.txt` — Enter the same passphrase. Verify the result. Run: `cat decrypted.txt` — The original message is restored.
 5. Generate an RSA private key for asymmetric encryption. Run: `openssl genrsa -out private.key 2048` — Extract the public key from it. Run: `openssl rsa -in private.key -pubout -out public.key`
 6. Compute the SHA-256 hash of a file to verify its integrity. Run: `openssl dgst -sha256 plaintext.txt` — Note the hash value. Make any change to the file and run the command again to see that the hash changes completely.
 7. Generate a GPG key pair. Run: `gpg --gen-key` — Enter your name and email when prompted, then set a passphrase. List the generated keys. Run: `gpg --list-keys`
 8. Encrypt the file using GPG with your own public key. Run: `gpg -e -r 'YourName' plaintext.txt` — This creates `plaintext.txt.gpg`. Decrypt it. Run: `gpg -d plaintext.txt.gpg` — Enter the passphrase to see the original content.
-

Conclusion

OpenSSL and GnuPG successfully encrypted and decrypted files using both symmetric AES-256 and asymmetric GPG encryption. Hash verification confirmed file integrity. This experiment demonstrates how encryption protects sensitive data during transmission and storage, forming the foundation of secure communication.